

Informe Desafio 1

Emmanuel Gonzalez Zabala
TI : 1036452245

Andres Felipe Orosco Posada
TI: 1025643969

Grupo : 4

Informática 2

Anibal Jose Guerra

Universidad de Antioquia

2026-1

El objetivo de este informe es presentar el análisis , lógica,y representación de datos empleadas en el desarrollo del desafío 1, el cual consiste en la realización de un **tetris** rudimentario en el lenguaje de programación C + +, y utilizando el *framework de Qt creator*.

TABLERO

En primera instancia debemos hablar de la estructura de datos que se usarán durante el desarrollo del programa. El cual se conforma de la siguiente forma:

fila 1	columna 1	columna 2	columna 3	...	...	...	columna m
fila 2							
fila 3							
...							
...							
...							
...							
fila n							

Siendo esta la estructura de datos del tablero (espacio de juego) del tetris, la cual será una **matriz totalmente dinámica** en la que cada uno de sus elementos son **apuntadores a char**, el uso de este tipo de variable es debido a que el tamaño del char en memoria es de 1 byte, este detalle resulta importante por los factores que veremos adelante tanto de Game Over, colisiones, representación de las figuras, etc.

Se decidió utilizar una matriz totalmente dinámica debido a que permite manejar el tablero mediante un apuntador de apuntadores (char **), lo que facilita el acceso y modificación de las celdas durante la ejecución del programa.

La desicion mas a fondo del uso de variables de tipo char para la creación del código, como se dijo anteriormente es debido a que estos ocupan 1 byte en memoria, lo cual va a facilitar a la hora de trabajar con operaciones a nivel de bits, por lo mismo resulta ser un requisito que la entrada del ancho sea un múltiplo de 8. Además cada fila se reserva dinámicamente, lo que permite manejar el tablero mediante un apuntador de apuntadores(char**), esto facilita el acceso y modificación de las celdas de los elementos y sus direcciones de

memoria, esto combinado con las operaciones a nivel de bits resultan ser una gran solución para los problemas de desplazamiento de las fichas y también la actualización del tablero.

PIEZAS O FICHAS (TETRIS)

Las fichas del juego se representarán mediante un entero de 16 bits(2 bytes) *unsigned short*

Esto permite poder modelar una matriz de 4x4 donde cada bit representa una celda de pieza.

Cada bit indica una celda ocupada (1) o cuando está vacía(0).

Representación conceptual de la lógica de las fichas (ejemplo la pieza T).

0	1	0	0
1	1	1	0
0	0	0	0
0	0	0	0

Cada posición corresponde exactamente a un bit dentro del entero de 16 bits. exactamente donde estan los (1) o llenos, ahí en representación visual se plasmará con los símbolos “#” y para representar las casillas vacías(0) se le asignará el símbolo “.”.

Se decidió representar las fichas mediante máscaras de bits por las siguientes razones.

- una matriz tradicional de char requería (4 x 4) bytes, mientras que esto se reduce gracias al unsigned short que son tan solo 2 bytes, por lo tanto el uso de máscaras minimiza significativamente el consumo de memoria.
- Las transformaciones de las piezas pueden hacerse mediante bitwise tales como:
 - & (AND)
 - | (OR)
 - << (SHIFT IZQUIERDA)
 - >> (SHIFT DERECHA)

Estas operaciones son muy rápidas a nivel de la cpu y además permite que no se ocupe un espacio significativo en la memoria pre calculando las rotaciones de las fichas.

- **ROTACIONES EN TIEMPO DE EJECUCIÓN** las rotaciones de las piezas no se almacenan previamente, sino que se calculan durante la ejecución del programa. esto permite:
 - Evitar almacenar múltiples versiones de la misma pieza
 - Mantener una estructura compacta

GENERACIÓN DE FICHAS

La problemática de la generación de las fichas se realizará de manera aleatoria mediante la utilización de la librería **<random>**, debido a que esta cuenta con funciones que permiten cumplir con la aleatoriedad.

Esto será posible diferenciando a cada ficha mediante un identificador. Por ejemplo, $T = 1$, $Z = 3$, etc. Así, la función **“uniform_int_distribution<int> dist()”** se encarga de elegir una entre los N identificadores (fichas) entregados.

Después de calcular cuál será la siguiente ficha, se realiza su impresión en pantalla y en la mitad del tablero, además, que salga poco a poco.

Se decidió realizar este apartado de acuerdo al ancho del tablero ingresado por el usuario, ya que la mitad de este sería simplemente $(\text{ancho} / 2)$, y al ser el ancho un múltiplo de 8 por requerimiento para poder trabajar a nivel de bits, entonces la mitad siempre será un número par, después la ficha se ubicará en la columna X , siendo X el resultado de la operación anterior.

La solución encontrada para poder resolver el problema de la aparición poco a poco de las fichas se concluyó con que la ficha siguiente aparezca línea a línea en pantalla a partir de ciclos for que van a recorrer el tablero en la fila i , y columnas j , donde el aumento será de 1 para ambos elementos (fila y columna) para poder imprimir la ficha fila a fila.

DESPLAZAMIENTO DE FICHAS

- **COLISIONES DE LA TABLA**

La detección de colisiones permite determinar si las piezas pueden desplazarse o rotarse dentro del tablero sin invadir espacios que ya están ocupados o salir de los límites del área de juego.

Para verificar una colisión, se recorre la matriz de las piezas evaluando cada uno de sus 16 bits. Cuando se detecta un bit activo (esto indica que la pieza ocupa esa celda), se calcula la posición correspondiente mediante las coordenadas de las piezas en el tablero.

Una vez obtenida esta posición, se verifican dos condiciones:

- Si la celda se encuentra fuera de los límites del tablero
- Si la celda del tablero ya se encuentra ocupada por un bloque previamente fijado.

● ROTACIONES DE FICHAS

Como las fichas son representadas en una matriz de 4x4 bits y el objetivo es rotarlas durante el tiempo de ejecución en 90° en sentido horario, entonces la estrategia es:

- Recorrer cada celda de la matriz 4x4
- Verificar si el bit está activo
- Calcular su nueva posición
- Activar este bit en la nueva pieza

Entonces para rotar las piezas durante la ejecución del programa la versión La forma más óptima para hacerlo es mediante operaciones a nivel de bits.

Para realizar la rotación de 90°, se aplica una transformación sobre las coordenadas de cada celda de la matriz, esto significa que si una celda está en la posición (i, j), su nueva posición después de la rotación será:

- $(i, j) \rightarrow (j, 3-i)$

Esta operación se lleva a cabo recorriendo la matriz y cada vez que encuentre un bit activo, a este se le calculará su nueva posición.

Este método, permite que las rotaciones sean calculadas solo cuando el usuario lo solicite, generando un sistema más flexible y eficiente.

● DESPLAZAMIENTOS HORIZONTALES Y VERTICALES

El movimiento de la pieza en una matriz lógica de 4x4 se logra cambiando la posición de la pieza dentro del tablero.

Para ello se utilizan dos coordenadas que indican la fila y la columna donde se encuentra.

Cuando el jugador intenta mover la ficha a la izquierda, a la derecha o abajo, Primero se calcula la nueva posición. Antes de aplicar el cambio, se verifica si en esa posición se genera una colisión con los bordes del tablero o con bloques activos que ya están fijados.

Al verificar si hay o no hay una colisión, entonces se actualiza el tablero con la nueva posición de la pieza. En caso de que se detecte una colisión se cancelará el movimiento. Para el caso de que la colisión sea hacia abajo la ficha se considerará fija y pasaría a formar parte del tablero.

Para determinar si las celdas de la pieza están ocupadas se usan las operaciones bitwise, verificando los bits activos de la máscara mediante operaciones AND(&) y desplazamientos(<<).

ELIMINACIÓN DE FILA

La solución a esta problemática se torna en las operaciones a nivel de bits y la primera declaración que se nombró, la cual fue que los espacios en el tablero serán de tipo char.

Se analizará cada vez que una ficha colisione y se revisa si la fila con la cual colisionó se llena por completo de unos (es decir que todos los elementos de esa fila están activos, por ende, ocupados), solo en ese entonces es que la fila se elimina, debido a que para simular el sistema de “eliminación y caída” se decidió que las filas se rellenaran con los elementos que tenía la fila de arriba.

Por ejemplo, si la última fila fue eliminada, entonces los elementos de la última fila se reemplazarán por los elementos de la penúltima fila, los elementos de la penúltima fila se reemplazarán por los elementos que estaban en la fila anterior a ella y así sucesivamente.

Para verificar si una fila está completamente llena se recorre cada una de sus celdas comprobando si todas contiene el carácter que representa que es un bloque activo, el cual verifica si la fila está activa o no (llena de (1)), se utiliza la máscara adecuada para el ancho del tablero, por ejemplo, si el tablero mide 8, la máscara será (0b11111111), y de ahí se comprueba mediante un ciclo anidado donde si se cumple con la condición, la fila será eliminada, y si no, se sigue iterando para revisar si hay alguna fila que pueda ser eliminada, si no hay ninguna fila llena, no se realiza ningún cambio, en caso contrario, aparte de ser eliminada la fila, se reemplazan los elementos de las filas.

GAME OVER

El sistema de terminación del juego es igual que en el Tetris original, es decir, cuando los bloques se acumulan hasta la parte superior del tablero, haciendo que no quepa otro bloque más.

Habrà un sistema de detección que indique cuando la ficha siguiente colisione con la primera fila, generando automáticamente el mensaje de GAME OVER.

El sistema se basa en un condicional usando el mismo sistema de colisión antes visto pero esta vez con la ficha siguiente y la primera fila, verificando si colisionan o no.

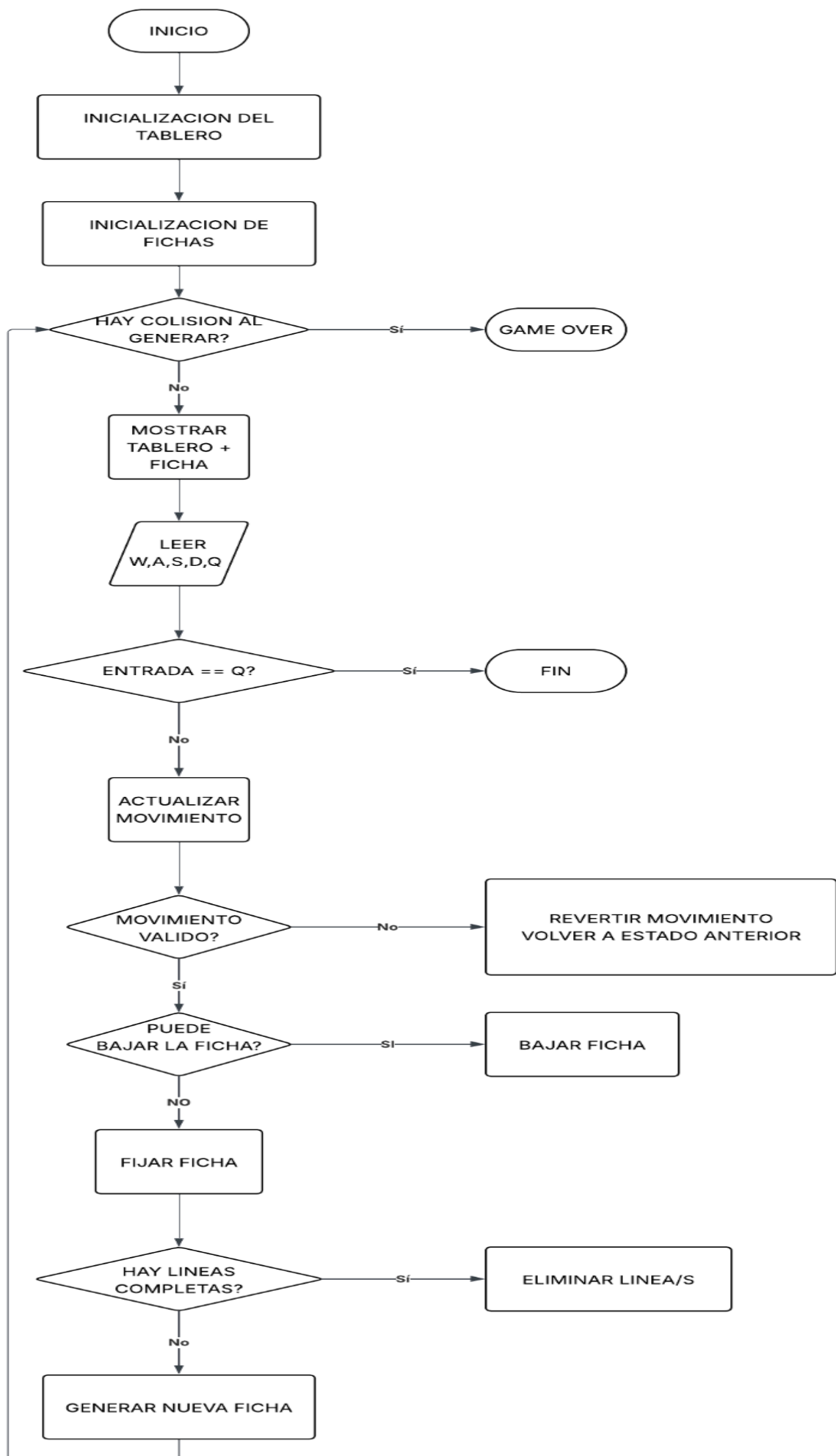
DIAGRAMA DE FLUJO(TETRIS)

- **EXPLICACIÓN DEL DIAGRAMA**

El diagrama de flujo presentado describe el funcionamiento general del juego. El proceso inicia con la generación de una ficha, seguida de su posicionamiento en el tablero.

Posteriormente, el sistema permite el movimiento y rotación de la ficha, verificando en cada paso si se producen colisiones. Cuando la ficha ya no puede descender más, esta se fija en el tablero y se procede a verificar si existen filas completas.

En caso de existir, estas son eliminadas y el tablero se actualiza. Este ciclo se repite continuamente hasta que se detecta una colisión en la parte superior del tablero, lo que indica el fin del juego.



ANÁLISIS DEL PROBLEMA

Entre las principales restricciones se encuentra la necesidad de manejar eficientemente la memoria, así como representar las piezas y el tablero de manera compacta. Además, se requiere que el sistema sea capaz de gestionar colisiones, rotaciones y eliminación de filas en tiempo de ejecución.

Debido a estas condiciones, se optó por una solución basada en el uso de matrices dinámicas y operaciones a nivel de bits, lo cual permite optimizar tanto el rendimiento como el uso de memoria.

- **ALGORITMOS IMPLEMENTADOS**

A continuación se describen los principales algoritmos utilizados en el desarrollo del programa:

- Rotación de fichas

La rotación de las fichas se realiza mediante una transformación de coordenadas dentro de una matriz de 4x4 bits. Para cada bit activo en la posición (i, j) , su nueva posición se calcula como $(j, 3 - i)$.

Este procedimiento se realiza en tiempo de ejecución, evitando almacenar múltiples versiones de cada ficha.

- Detección de colisiones

Para determinar si una ficha puede moverse, se recorre su representación binaria y se evalúan las posiciones correspondientes en el tablero.

Si alguna de estas posiciones se encuentra fuera de los límites o ya está ocupada, se considera que existe una colisión.

- Eliminación de filas

El algoritmo de eliminación de filas verifica si todas las celdas de una fila están ocupadas. En caso afirmativo, se eliminan y las filas superiores descienden una posición.

Este proceso se repite hasta que no existan más filas completas.

- **PROBLEMAS DE DESARROLLO (CLAVE)**

Durante el desarrollo del proyecto se presentaron diversas dificultades. Una de las principales fue la implementación de las rotaciones mediante operaciones bitwise, ya que inicialmente no se obtenían los resultados esperados.

También se presentaron errores en la detección de colisiones, donde algunas fichas lograban atravesar otras debido a fallos en el cálculo de posiciones.

Pero los principales problemas afrontados durante la realización del código fueron más que todo interferencias e inconvenientes entre la estructura de datos del tablero y las fichas, consiguiendo que hayan fallas a la hora de la realización del programa principal. Inconvenientes tales como los tipos de las variables, las lecturas, el recorrido por el tablero, los límites y las colisiones.

Estos problemas se pudieron solucionar al realizar el programa principal y ver a fondo las interferencias mediante la realización de pruebas de cada una de las funcionalidades del juego y ver cómo reaccionaba.

Adicionalmente, la integración del trabajo entre los integrantes generó inconvenientes, especialmente al momento de unificar las distintas partes del código, lo que requirió ajustes y pruebas adicionales.

- **EVOLUCIÓN DE LA SOLUCIÓN**

Inicialmente, se planteó representar las fichas mediante matrices tradicionales de tipo char, sin embargo, esta solución resultaba poco eficiente en términos de memoria.

Posteriormente, se decidió utilizar una representación basada en enteros de 16 bits, lo que permitió reducir significativamente el espacio utilizado y mejorar la velocidad de las operaciones.

De igual forma, las rotaciones pasaron de ser consideradas como valores predefinidos a calcularse dinámicamente en tiempo de ejecución, lo que aportó mayor flexibilidad al sistema.

Las funciones del módulo del tablero en las que se usaban la pieza se creía que se podían usar algo como unsigned char[4] pero esto no concordaba con lo que hacía pieza por ende se debían crear una función de transformación de estructura de datos, lo que era innecesario e ineficiente

Además la función de eliminación de fila se planteó hacerla con bool para ver si había una fila llena o una función sin bool simplemente recorriendo con condicionales en este caso nos acomodamos con la versión sin usar bool

- **CONCLUSIONES**

El desarrollo del proyecto permitió comprender la importancia de la optimización en el uso de memoria y el manejo eficiente de estructuras de datos.

El uso de operaciones a nivel de bits demostró ser una herramienta poderosa para la representación y manipulación de información de forma compacta.

Finalmente, el trabajo en equipo y la resolución de problemas durante el desarrollo fueron factores clave para lograr una solución funcional del juego Tetris.